

Automated EC2 Health Monitoring & Multi-Channel Alerts

PROBLEM

When managing multiple EC2 instances, manually checking every server is time-consuming. If an instance stops or has a health issue, it can easily go unnoticed.

HOW I DEBUGGED IT

I wanted to avoid checking each instance manually, so I automated the health checks. I monitor the Running/Stopped state, 1/2 Status Checks, CPU, Memory, and Disk utilization across all EC2 instances.

THE SOLUTION

EventBridge → Lambda → EC2 / CloudWatch / SSM → Slack

Every 5 minutes, Lambda checks all EC2 instances. If an instance stops or any issue is detected, it automatically sends the details to Slack.

This helps me quickly identify which server has the problem, what went wrong, and where to start troubleshooting, without manually checking every instance.

Result: Less manual monitoring, faster issue detection, and quicker troubleshooting.

Has anyone else faced this problem?

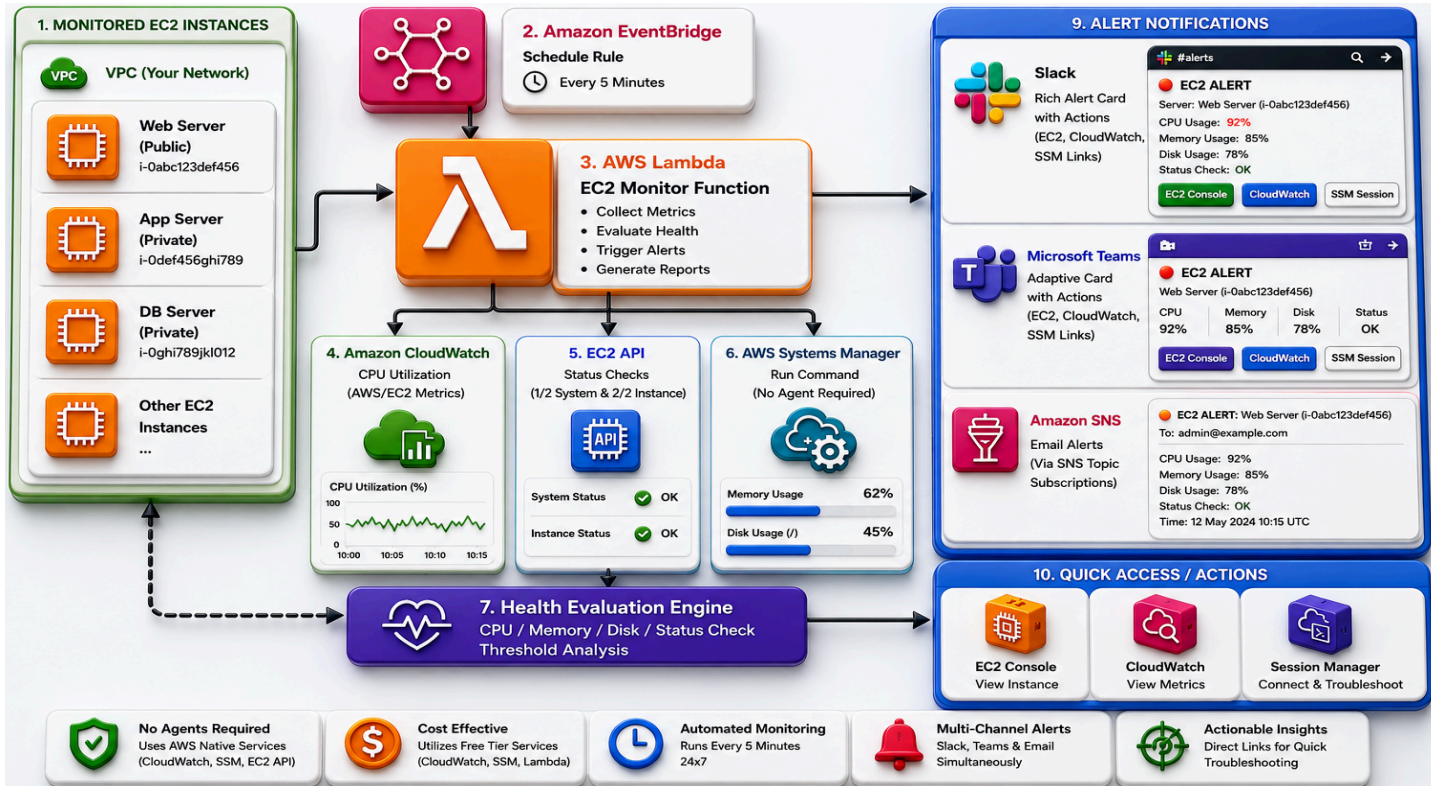
How did you fix it?

Note: The complete code is available on the last page for reference.

Blog: <https://vishnusai.hashnode.dev/automated-ec2-health-monitoring-multi-channel-alerts>

HashnodProfile: <https://hashnode.com/@vishnusaiK>

GitHub : <https://github.com/ksaivishnusaiKvs>



INFRA Upgrade subscription

Find a conversation...

Slackbot, Huddles, Drafts & sent, Directories, Starred, Channels, Direct messages, Agents and apps

New message Saved 5 minutes ago

To: dbalerts

Alerts APP 15:37

[PROD] EC2 ALERT – testing (i-00430b1a074d82e2c)

- Status Check (1/2 System): **not-applicable** (threshold: ok) [CRITICAL] – AWS hardware issue – may require stop/start to migrate host
- Status Check (2/2 Instance): **not-applicable** (threshold: ok) [CRITICAL] – OS/kernel issue – check system logs or reboot

Instance	Name
i-00430b1a074d82e2c	testing
State	Region
stopped	eu-north-1

Current metrics

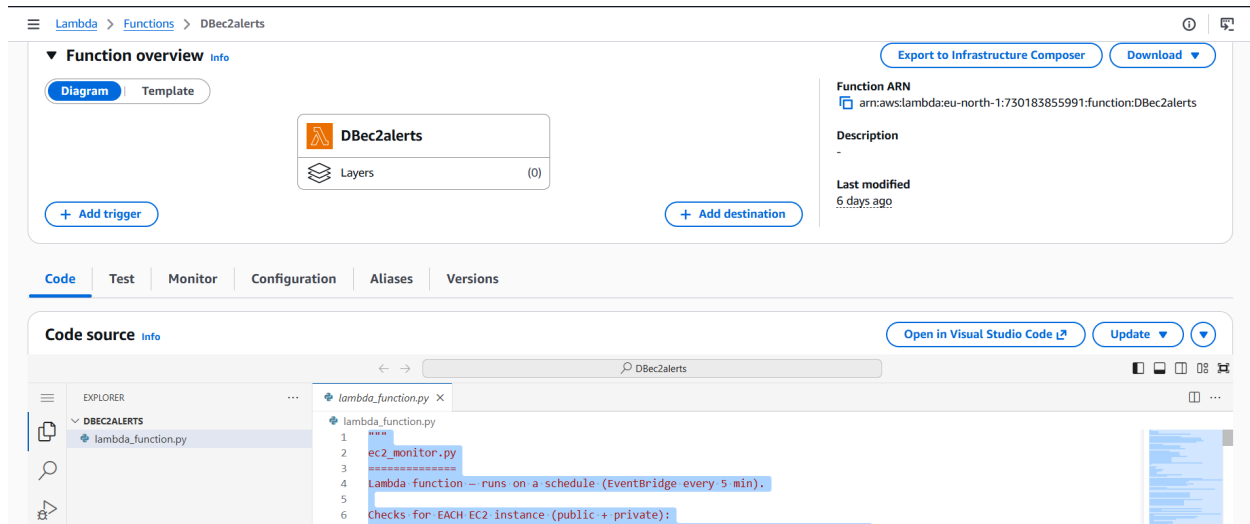
CPU: N/A | Mem: N/A (no SSM) | Disk: N/A (no SSM) | Status: not-applicable/not-applicable

Time: 2026-07-07 10:07:14 UTC | Env: PROD

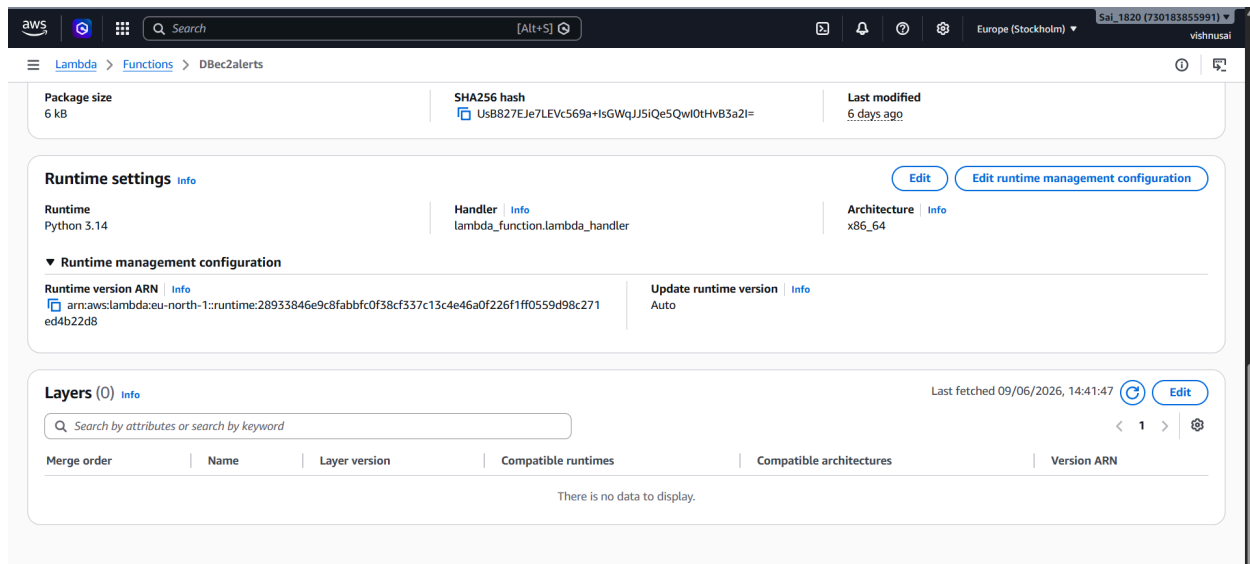
[EC2 Console](#) [CloudWatch](#) [SSM Run Command](#)

[Show less](#)

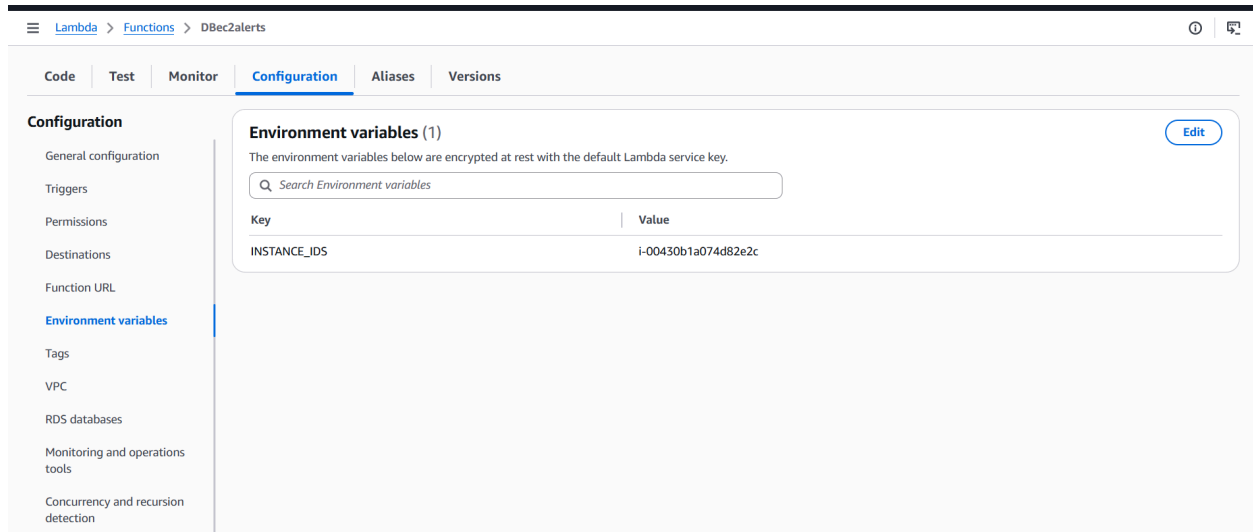
Step 1: Create the Lambda function to perform EC2 health checks. Configure the function with the required Python runtime and prepare it to execute the monitoring code.



Step 2: Configure Lambda. Configure the Lambda function with the required environment variables, including EC2 instance IDs, AWS Region, alert thresholds, Slack/Teams webhook URLs, and SNS topic ARN.

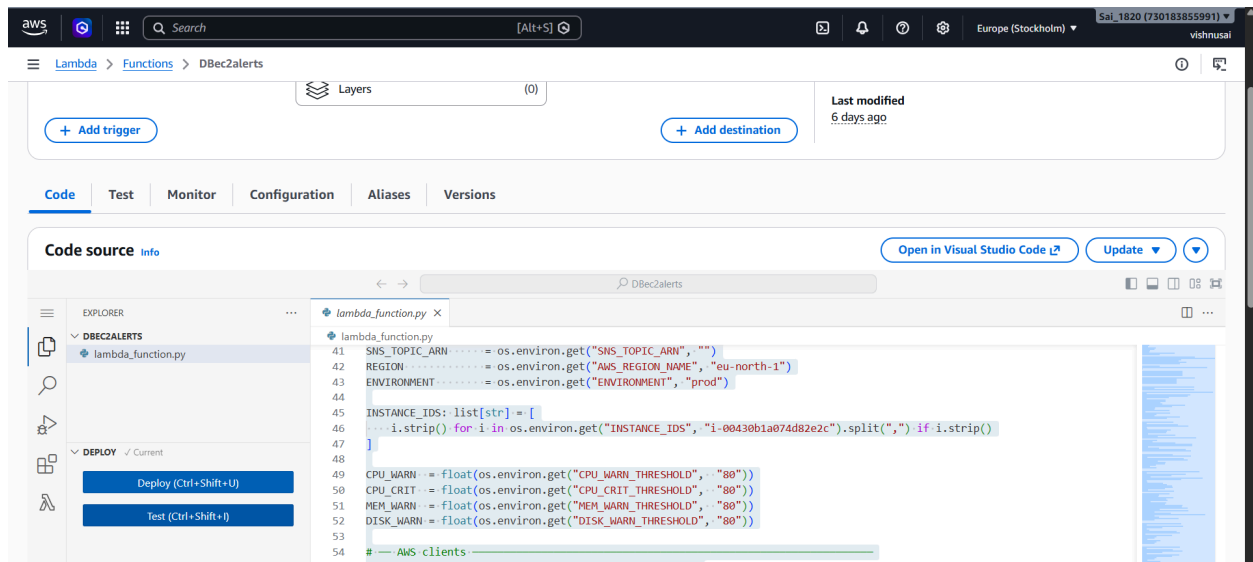


Step 3: Configure the Lambda function with the required environment variables, including EC2 instance IDs



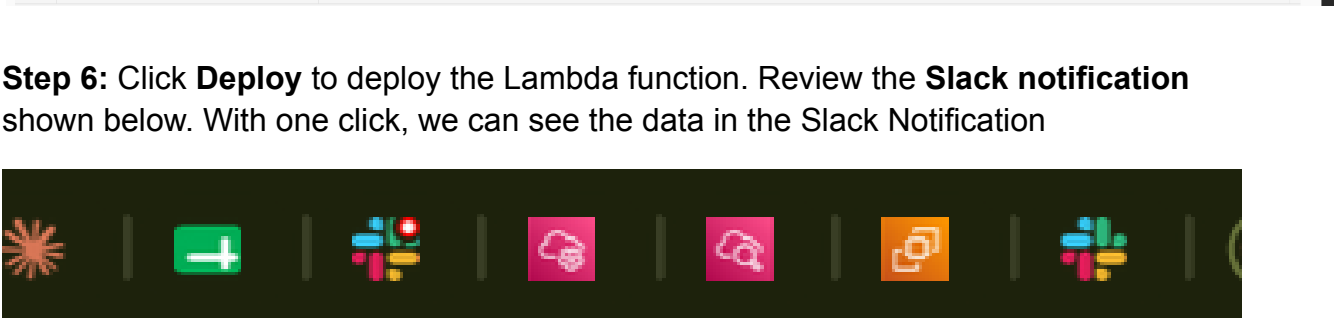
Step 4: Deploy the Lambda function and run a test execution. Verify that the function can successfully collect the EC2 health information and return the expected response

Now we test and deploy



The screenshot shows the AWS Lambda console interface. On the left, the 'EXPLORER' pane shows the project structure with 'DBEC2ALERTS' and 'lambda_function.py'. The 'DEPLOY' section shows 'Current' as the selected version. The main pane displays the 'Execution Results' for the function. The status is 'Succeeded' and the test event name is 'devops'. The response is a JSON object:

```
{
  "statusCode": 200,
  "summary": {
    "checked": 1,
    "healthy": 0,
    "alerted": 1,
    "timestamp": "2026-06-09T09:27:16.126709+00:00",
    "results": [
      {
        "id": "i-00430b1a074d82e2c",
        "name": "function"
      }
    ]
  }
}
```



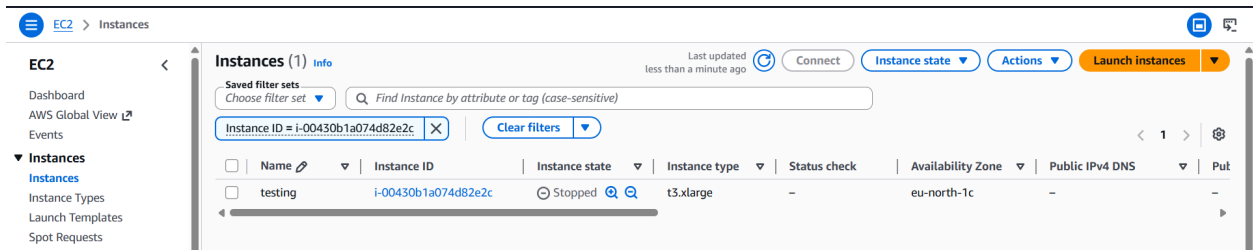
Step 7: Receive the EC2 alert in Slack with clear information about the alert/warning, instance ID, server name, region, status, and timestamp.

The screenshot shows a Slack workspace named 'INFRA'. The left sidebar contains navigation options: Home, DMs, Activity, Files, and Admin. The main channel is '#dbalerts'. A message from the 'Alerts' app is displayed, titled '[PROD] EC2 ALERT – testing (i-00430b1a074d82e2c)'. The message contains two status checks, both marked as 'not-applicable' with a 'threshold: ok' and '[CRITICAL]' status. The first check is for the 'System' (AWS hardware issue) and the second is for the 'Instance' (OS/kernel issue). Below the checks is a table with instance details:

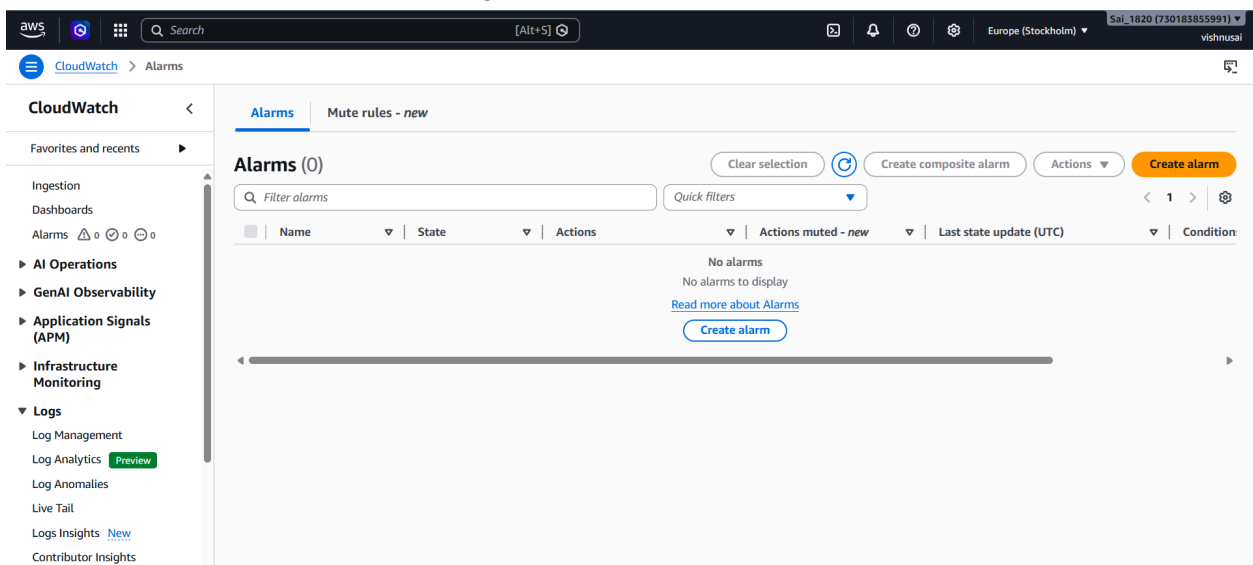
Instance	Name
i-00430b1a074d82e2c	testing
State	Region
stopped	eu-north-1

Below the table, the 'Current metrics' are shown: CPU: N/A, Mem: N/A (no SSM), Disk: N/A (no SSM), and Status: not-applicable/not-applicable. The message also includes a timestamp 'Time: 2026-06-09 09:27:15 UTC | Env: PROD' and three buttons: 'EC2 Console', 'CloudWatch', and 'SSM Run Command'. The message is followed by a 'Show less' link. The bottom of the interface shows the Slack message input area with various formatting and action icons.

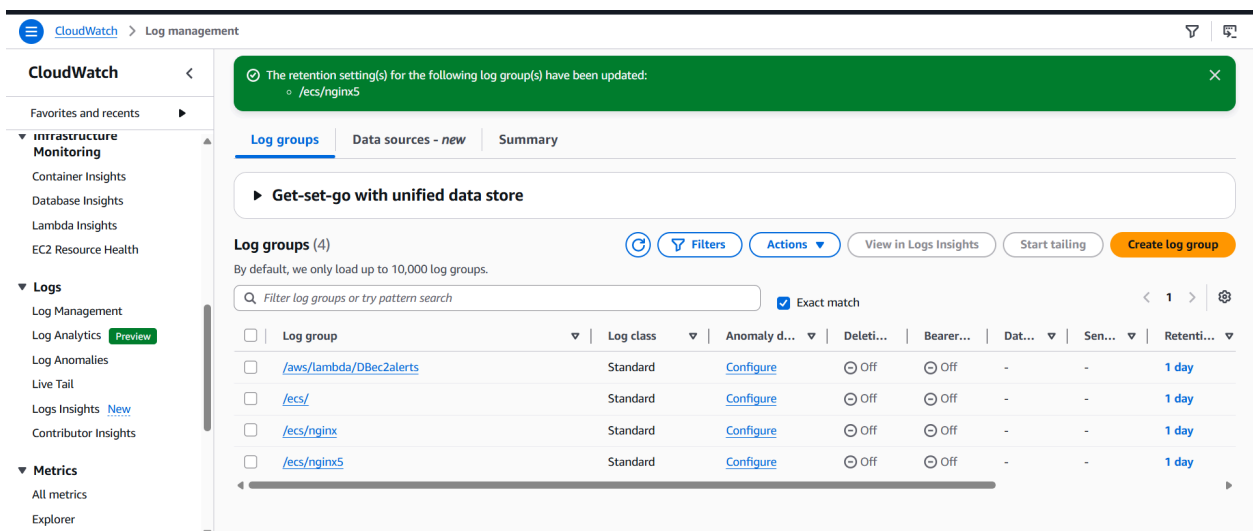
Step 8: Click on the navigation buttons for the EC2 Console. Check the EC2 instance status to confirm whether the server is Running, Stopped, or in a transitional state before performing the recovery action.



➤ Click on the CloudWatch navigation button



➤ Navigate to the CloudWatch logs



SessionManager

The screenshot displays the AWS Session Manager console. At the top, the AWS navigation bar includes the logo, a search bar, and user information (Sai_1820, 730183855991). The left-hand navigation pane is titled 'AWS Systems Manager' and lists various tools, with 'Session Manager' highlighted under the 'Node Tools' section. The main content area features a dark header with the title 'Session Manager' and the subtitle 'Quickly and securely access your Windows and Linux instances'. Below this, a brief description states that Session Manager is a managed service for secure, one-click access to instances without the need for inbound ports or bastion hosts. To the right of the description, there are two prominent white boxes: 'Start a session' with a 'Start Session' button and 'Configure Preferences' link, and 'Getting started' with a list of links including 'What is Session Manager?', 'Set up Session Manager', 'Set up session logging', 'Set up session notifications', 'Create and manage sessions', and 'Monitor session activity'. A 'How it works' section is also present, detailing four steps: 1. Configure your instances to use Session Manager, 2. Assign user IAM policies to control instance access, 3. Specify account options for session logs, and 4. Start a session on your instances by launching bash or shell terminal. The footer of the console shows links for 'CloudShell', 'Feedback', and 'Console Mobile App', along with the copyright notice '© 2026, Amazon Web Services, Inc. or its affiliates.' and links for 'Privacy', 'Terms', and 'Cookie preferences'.

aws [Search] [Alt+S]

Europe (Stockholm) Sai_1820 (730183855991) vishrusai

AWS Systems Manager

- Review node insights
- Explore nodes
- Diagnose and remediate
- Just-in-time node access [New](#)
- Settings

▼ Node Tools

- Compliance
- Distributor
- Fleet Manager
- Hybrid Activations
- Inventory
- Patch Manager
- Run Command
- Session Manager**
- State Manager

▼ Change Management Tools

Session Manager

Management

Quickly and securely access your Windows and Linux instances

Session Manager is a managed service that provides you with one-click secure access to your instances without the need to open inbound ports and manage bastion hosts. You have centralized access control over who can access your instances and full auditing capabilities to ensure compliance with corporate policies.

Start a session

Connect to your instances by starting a secure and auditable session.

[Start Session](#)

[Configure Preferences](#)

Getting started

- [What is Session Manager?](#)
- [Set up Session Manager](#)
- [Set up session logging](#)
- [Set up session notifications](#)
- [Create and manage sessions](#)
- [Monitor session activity](#)

How it works

- 1 Configure your instances to use Session Manager
- 2 Assign user IAM policies to control instance access
- 3 Specify account options for session logs
- 4 Start a session on your instances by launching bash or shell terminal

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

REF IMAGES

app.slack.com/client/T0A2UEHQM62/C0B7YDR6789

Search INFRA

dbalerts

Upgrade subscription

Home

Huddles

Drafts & sent

Directories

Starred

Drag and drop important stuff here

Channels

dbalerts

infra

montring

Direct messages

Apps

Messages

Add canvas

Wednesday, 3 June

Today

Alerts APP 14:57

[PROD] EC2 ALERT - testing (i-00430b1a074d82e2c)

- Status Check (1/2 System): not-applicable (threshold: ok) [CRITICAL] - AWS hardware issue - may require stop/start to migrate host
- Status Check (2/2 Instance): not-applicable (threshold: ok) [CRITICAL] - OS/kernel issue - check system logs or reboot

Instance	Name
i-00430b1a074d82e2c	testing
State	Region
stopped	eu-north-1

Current metrics

CPU: N/A | Mem: N/A (no SSM) | Disk: N/A (no SSM) | Status: not-applicable/not-

Search INFRA

dbalerts

Upgrade subscription

Home

Huddles

Drafts & sent

Directories

Starred

Drag and drop important stuff here

Channels

dbalerts

infra

montring

Direct messages

Apps

Messages

Add canvas

Today

Status Check (1/2 System): not-applicable (threshold: ok) [CRITICAL] - AWS hardware issue - may require stop/start to migrate host

Status Check (2/2 Instance): not-applicable (threshold: ok) [CRITICAL] - OS/kernel issue - check system logs or reboot

Instance	Name
i-00430b1a074d82e2c	testing
State	Region
stopped	eu-north-1

Current metrics

CPU: N/A | Mem: N/A (no SSM) | Disk: N/A (no SSM) | Status: not-applicable/not-applicable

Time: 2024-06-09 09:27:15 UTC | Env: PROD

EC2 Console CloudWatch SSM Run Command

Show less

Ec2_monitor.py

=====

Lambda function — runs on a schedule (EventBridge every 5 min).

Checks for EACH EC2 instance (public + private):

1. CPU utilization — CloudWatch metric AWS/EC2 (always free, no agent)
2. Status checks 1/2 & 2/2 — CloudWatch metric AWS/EC2 (always free, no agent)
3. Memory utilization — SSM Run Command reads /proc/meminfo (no agent needed)
4. Disk utilization — SSM Run Command reads df (no agent needed)

Sends rich alerts to:

- Slack (Block Kit cards with action buttons)
- Teams (Adaptive Cards with action buttons)
- Email (via SNS — already subscribed in your infra)

Environment variables required:

SLACK_WEBHOOK_URL Slack incoming webhook
TEAMS_WEBHOOK_URL Teams incoming webhook
SNS_TOPIC_ARN SNS topic ARN for email
INSTANCE_IDS Comma-separated: i-aaa,i-bbb,i-ccc
CPU_WARN_THRESHOLD Default 80
CPU_CRIT_THRESHOLD Default 95
MEM_WARN_THRESHOLD Default 85
DISK_WARN_THRESHOLD Default 85
AWS_REGION_NAME e.g. ap-south-1 (note: not AWS_REGION — that's reserved)

"""

```
import json
import os
import urllib.request
import urllib.error
import boto3
import time
from datetime import datetime, timedelta, timezone
from typing import Optional
```

— Config from environment

```
SLACK_WEBHOOK_URL = os.environ.get("SLACK_WEBHOOK_URL",
"https://hooks.slack.com/services/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Y")
TEAMS_WEBHOOK_URL = os.environ.get("TEAMS_WEBHOOK_URL", "")
```

```

SNS_TOPIC_ARN    = os.environ.get("SNS_TOPIC_ARN", "")
REGION           = os.environ.get("AWS_REGION_NAME", "eu-north-1")
ENVIRONMENT      = os.environ.get("ENVIRONMENT", "prod")

INSTANCE_IDS: list[str] = [
    i.strip() for i in os.environ.get("INSTANCE_IDS", "i-00430b1a074d82e2c").split(",") if i.strip()
]

```

```

CPU_WARN = float(os.environ.get("CPU_WARN_THRESHOLD", "80"))
CPU_CRIT = float(os.environ.get("CPU_CRIT_THRESHOLD", "80"))
MEM_WARN = float(os.environ.get("MEM_WARN_THRESHOLD", "80"))
DISK_WARN = float(os.environ.get("DISK_WARN_THRESHOLD", "80"))

```

```

# — AWS clients

```

```

cw = boto3.client("cloudwatch", region_name=REGION)
ec2 = boto3.client("ec2", region_name=REGION)
ssm = boto3.client("ssm", region_name=REGION)
sns = boto3.client("sns", region_name=REGION)

```

```

#

```

```

# METRIC COLLECTORS

```

```

#

```

```

def get_cloudwatch_metric(instance_id: str, metric_name: str, namespace: str = "AWS/EC2") ->
Optional[float]:

```

```

    """

```

```

    Fetch the latest average value of a CloudWatch metric for an instance.

```

```

    Uses the last 10 minutes window to get the most recent data point.

```

```

    Returns None if no data is available.

```

```

    """

```

```

    now = datetime.now(timezone.utc)

```

```

    resp = cw.get_metric_statistics(

```

```

        Namespace=namespace,

```

```

        MetricName=metric_name,

```

```

        Dimensions=[{"Name": "InstanceId", "Value": instance_id}],

```

```

        StartTime=now - timedelta(minutes=10),

```

```

        EndTime=now,

```

```

        Period=300,
        Statistics=["Average", "Maximum"],
    )
    datapoints = resp.get("Datapoints", [])
    if not datapoints:
        return None
    latest = max(datapoints, key=lambda d: d["Timestamp"])
    return round(latest["Average"], 2)

```

```
def get_status_checks(instance_id: str) -> dict:
```

```

    """
    Returns status check results directly from EC2 API.
    status = 'ok' | 'impaired' | 'insufficient-data' | 'not-applicable'
    """
    try:
        resp = ec2.describe_instance_status(
            InstanceIds=[instance_id],
            IncludeAllInstances=True,
        )
        statuses = resp.get("InstanceStatuses", [])
        if not statuses:
            return {"system": "unknown", "instance": "unknown", "state": "unknown"}

        s = statuses[0]
        return {
            "state": s["InstanceState"]["Name"],
            "system": s["SystemStatus"]["Status"], # ok | impaired | ...
            "instance": s["InstanceStatus"]["Status"], # ok | impaired | ...
        }
    except Exception as e:
        print(f"[WARN] Status check failed for {instance_id}: {e}")
        return {"system": "error", "instance": "error", "state": "unknown"}

```

```
def get_instance_name(instance_id: str) -> str:
```

```

    """Return the Name tag of an instance, or the instance ID if no tag."""
    try:
        resp = ec2.describe_instances(InstanceIds=[instance_id])
        tags = resp["Reservations"][0]["Instances"][0].get("Tags", [])
        for tag in tags:
            if tag["Key"] == "Name":
                return tag["Value"]
    except Exception:

```

```
    pass
    return instance_id
```

```
def run_ssm_command(instance_id: str, command: str, timeout: int = 20) -> Optional[str]:
```

```
    """
```

Execute a shell command on an EC2 instance via SSM Run Command.

Returns stdout string or None on failure.

Requires:

- EC2 instance has SSM Agent running (pre-installed on Amazon Linux 2/2023, Ubuntu 16+)
- EC2 IAM role has AmazonSSMManagedInstanceCore policy

```
    """
```

try:

```
    resp = ssm.send_command(
        InstanceIds=[instance_id],
        DocumentName="AWS-RunShellScript",
        Parameters={"commands": [command]},
        TimeoutSeconds=timeout,
    )
    command_id = resp["Command"]["CommandId"]
```

Poll for result (max 25 seconds)

```
    for _ in range(5):
        time.sleep(5)
        result = ssm.get_command_invocation(
            CommandId=command_id,
            InstanceId=instance_id,
        )
        status = result["Status"]
        if status == "Success":
            return result["StandardOutputContent"].strip()
        if status in ("Failed", "Cancelled", "TimedOut"):
            print(f"[WARN] SSM command {status} on {instance_id}")
            return None
    print(f"[WARN] SSM command timed out polling on {instance_id}")
    return None
```

except ssm.exceptions.InvalidInstanceId:

```
    print(f"[WARN] {instance_id} not registered with SSM (agent not running or no IAM role)")
    return None
```

except Exception as e:

```
    print(f"[WARN] SSM error on {instance_id}: {e}")
    return None
```

```
def get_memory_via_ssm(instance_id: str) -> Optional[float]:
    """
    Read memory usage % from /proc/meminfo via SSM Run Command.
    No CloudWatch Agent needed.
    """
    cmd = (
        "awk '/MemTotal/{t=$2} /MemAvailable/{a=$2} "
        "END{printf \"%.2f\", (t-a)/t*100}' /proc/meminfo"
    )
    output = run_ssm_command(instance_id, cmd)
    if output is None:
        return None
    try:
        return float(output)
    except ValueError:
        return None
```

```
def get_disk_via_ssm(instance_id: str) -> Optional[float]:
    """
    Read root disk usage % via SSM Run Command.
    Returns the highest used % across all mounted filesystems.
    No CloudWatch Agent needed.
    """
    cmd = "df -h --output=pcent / | tail -1 | tr -d ' '"
    output = run_ssm_command(instance_id, cmd)
    if output is None:
        return None
    try:
        return float(output)
    except ValueError:
        return None
```

```
#
```

```
# HEALTH EVALUATOR
#
```

```
def evaluate_instance(instance_id: str) -> dict:
    """
```

Collect all metrics for one instance and return a health report dict.

"""

```
name = get_instance_name(instance_id)
print(f" Checking {name} ({instance_id})...")
```

```
cpu   = get_cloudwatch_metric(instance_id, "CPUUtilization")
status = get_status_checks(instance_id)
memory = get_memory_via_ssm(instance_id)
disk   = get_disk_via_ssm(instance_id)
```

```
alerts = []
```

```
# CPU
```

```
if cpu is not None:
```

```
    if cpu >= CPU_CRIT:
```

```
        alerts.append({"metric": "CPU", "value": cpu, "threshold": CPU_CRIT, "severity":
"CRITICAL"})
```

```
    elif cpu >= CPU_WARN:
```

```
        alerts.append({"metric": "CPU", "value": cpu, "threshold": CPU_WARN, "severity":
"WARNING"})
```

```
# Memory
```

```
if memory is not None:
```

```
    if memory >= MEM_WARN:
```

```
        alerts.append({"metric": "Memory", "value": memory, "threshold": MEM_WARN,
"severity": "WARNING"})
```

```
# Disk
```

```
if disk is not None:
```

```
    if disk >= DISK_WARN:
```

```
        alerts.append({"metric": "Disk", "value": disk, "threshold": DISK_WARN, "severity":
"WARNING"})
```

```
# Status checks
```

```
if status["system"] not in ("ok", "unknown"):
```

```
    alerts.append({
```

```
        "metric": "Status Check (1/2 System)",
```

```
        "value": status["system"],
```

```
        "threshold": "ok",
```

```
        "severity": "CRITICAL",
```

```
        "note": "AWS hardware issue — may require stop/start to migrate host"
```

```
    })
```

```
if status["instance"] not in ("ok", "unknown"):
```

```
    alerts.append({
```

```

        "metric": "Status Check (2/2 Instance)",
        "value": status["instance"],
        "threshold": "ok",
        "severity": "CRITICAL",
        "note": "OS/kernel issue — check system logs or reboot"
    })

return {
    "instance_id": instance_id,
    "name": name,
    "state": status.get("state", "unknown"),
    "metrics": {
        "cpu": cpu,
        "memory": memory,
        "disk": disk,
        "status_system": status["system"],
        "status_instance": status["instance"],
    },
    "alerts": alerts,
    "healthy": len(alerts) == 0,
}

#
=====
# NOTIFICATION BUILDERS
#
=====

SEVERITY_COLORS = {
    "CRITICAL": {"hex": "#D32F2F", "teams": "attention", "emoji": "🔴"},
    "WARNING": {"hex": "#F57C00", "teams": "warning", "emoji": "🟡"},
    "OK": {"hex": "#388E3C", "teams": "good", "emoji": "✅"},
}

def _metric_line(report: dict) -> str:
    m = report["metrics"]
    cpu_str = f"{m['cpu']:.1f}%" if m["cpu"] is not None else "N/A"
    mem_str = f"{m['memory']:.1f}%" if m["memory"] is not None else "N/A (no SSM)"
    disk_str = f"{m['disk']:.1f}%" if m["disk"] is not None else "N/A (no SSM)"
    sys_str = m["status_system"]
    ins_str = m["status_instance"]

```



```

return (
    f"CPU: {cpu_str} | Mem: {mem_str} | "
    f"Disk: {disk_str} | Status: {sys_str}/{ins_str}"
)

```

```

def build_slack_message(report: dict) -> dict:

```

```

    instance_id = report["instance_id"]
    name       = report["name"]
    alerts     = report["alerts"]
    env        = ENVIRONMENT.upper()
    region     = REGION

```

```

    top_severity = "CRITICAL" if any(a["severity"] == "CRITICAL" for a in alerts) else "WARNING"
    color = SEVERITY_COLORS[top_severity]["hex"]
    emoji = SEVERITY_COLORS[top_severity]["emoji"]

```

```

    header = f"{emoji} [{env}] EC2 ALERT — {name} ({instance_id})"

```

```

    alert_lines = []

```

```

    for a in alerts:

```

```

        note = f" — {a['note']}" if "note" in a else ""

```

```

        alert_lines.append(

```

```

            f"• {a['metric']}: {a['value']} (threshold: {a['threshold']}) [{a['severity']}] {note}"

```

```

        )

```

```

    ec2_url =

```

```

    f"https://{region}.console.aws.amazon.com/ec2/home?region={region}#Instances:instanceId={instance_id}"

```

```

    cw_url =

```

```

    f"https://{region}.console.aws.amazon.com/cloudwatch/home?region={region}#alarmsV2:"

```

```

    logs_url =

```

```

    f"https://{region}.console.aws.amazon.com/systems-manager/run-command?region={region}"

```

```

    blocks = [

```

```

        {

```

```

            "type": "header",

```

```

            "text": {"type": "plain_text", "text": header[:150], "emoji": True}

```

```

        },

```

```

        {

```

```

            "type": "section",

```

```

            "text": {

```

```

                "type": "mrkdown",

```

```

                "text": "\n".join(alert_lines)

```

```

    }
  },
  {"type": "divider"},
  {
    "type": "section",
    "fields": [
      {"type": "mrkdwn", "text": f"*Instance*\n`{instance_id}`"},
      {"type": "mrkdwn", "text": f"*Name*\n{name}"},
      {"type": "mrkdwn", "text": f"*State*\n{report['state']}"},
      {"type": "mrkdwn", "text": f"*Region*\n{region}"},
    ]
  },
  {
    "type": "section",
    "text": {
      "type": "mrkdwn",
      "text": f"*Current metrics*\n`{_metric_line(report)}`"
    }
  },
  {
    "type": "context",
    "elements": [
      {"type": "mrkdwn", "text": f"Time: {datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M:%S')} UTC | Env: {env}"}
    ]
  },
  {
    "type": "actions",
    "elements": [
      {
        "type": "button",
        "text": {"type": "plain_text", "text": "EC2 Console"},
        "url": ec2_url,
        "style": "danger"
      },
      {
        "type": "button",
        "text": {"type": "plain_text", "text": "CloudWatch"},
        "url": cw_url
      },
      {
        "type": "button",
        "text": {"type": "plain_text", "text": "SSM Run Command"},
        "url": logs_url
      }
    ]
  }

```

```

        },
    ]
}
]

return {
    "attachments": [{
        "color": color,
        "blocks": blocks,
        "fallback": header
    }]
}

```

```
def build_teams_message(report: dict) -> dict:
```

```

    instance_id = report["instance_id"]
    name        = report["name"]
    alerts      = report["alerts"]
    env         = ENVIRONMENT.upper()
    region      = REGION

```

```

    top_severity = "CRITICAL" if any(a["severity"] == "CRITICAL" for a in alerts) else "WARNING"
    sev          = SEVERITY_COLORS[top_severity]
    title        = f'{sev["emoji"]} [{env}] EC2 ALERT — {name} ({instance_id})'

```

```

    ec2_url =
f"https://{region}.console.aws.amazon.com/ec2/home?region={region}#Instances:instanceId={instance_id}"
    cw_url =
f"https://{region}.console.aws.amazon.com/cloudwatch/home?region={region}#alarmsV2:"

```

```

    alert_facts = []
    for a in alerts:
        note = f" ({a.get('note', '')})" if "note" in a else ""
        alert_facts.append({
            "title": f'{a["severity"]} — {a["metric"]}',
            "value": f'{a["value"]} (threshold: {a["threshold"]}){note}'
        })

```

```
m = report["metrics"]
```

```

return {
    "type": "message",
    "attachments": [{

```

```

"contentType": "application/vnd.microsoft.card.adaptive",
"content": {
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "type": "AdaptiveCard",
  "version": "1.4",
  "body": [
    {
      "type": "TextBlock",
      "text": title,
      "weight": "bolder",
      "size": "medium",
      "wrap": True,
      "color": sev["teams"]
    },
    {
      "type": "FactSet",
      "facts": alert_facts
    },
    {"type": "TextBlock", "text": "Current metrics", "weight": "bolder", "size": "small"},
    {
      "type": "FactSet",
      "facts": [
        {"title": "CPU", "value": f"{m['cpu']:.1f}%" if m["cpu"] is not None else
"N/A"},
        {"title": "Memory", "value": f"{m['memory']:.1f}%" if m["memory"] is not None
else "N/A (no SSM)"},
        {"title": "Disk", "value": f"{m['disk']:.1f}%" if m["disk"] is not None else "N/A
(no SSM)"},
        {"title": "Status", "value": f"System: {m['status_system']} | Instance:
{m['status_instance']}"},
        {"title": "State", "value": report["state"]},
        {"title": "Region", "value": region},
        {"title": "Time", "value": datetime.now(timezone.utc).strftime("%Y-%m-%d
%H:%M:%S") + " UTC"},
      ]
    }
  ],
  "actions": [
    {"type": "Action.OpenUrl", "title": "Open EC2 Console", "url": ec2_url},
    {"type": "Action.OpenUrl", "title": "View CloudWatch", "url": cw_url},
  ]
}
}]
}

```

```

def build_sns_email(report: dict) -> str:
    """Plain text email body for SNS."""
    env = ENVIRONMENT.upper()
    name = report["name"]
    iid = report["instance_id"]
    lines = [
        f"[{env}] EC2 ALERT — {name} ({iid})",
        f"Time: {datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M:%S')} UTC",
        f"Region: {REGION}",
        f"State: {report['state']}",
        "",
        "ALERTS:",
    ]
    for a in report["alerts"]:
        note = f" | Note: {a.get('note', '')}" if "note" in a else ""
        lines.append(f" [{a['severity']}] {a['metric']}: {a['value']} (threshold: {a['threshold']}){note}")
    m = report["metrics"]
    lines += [
        "",
        "CURRENT METRICS:",
        f" CPU: {m['cpu']:.1f}%" if m["cpu"] is not None else " CPU: N/A",
        f" Memory: {m['memory']:.1f}%" if m["memory"] is not None else " Memory: N/A (SSM not available)",
        f" Disk: {m['disk']:.1f}%" if m["disk"] is not None else " Disk: N/A (SSM not available)",
        f" Status: System={m['status_system']} | Instance={m['status_instance']}",
        "",
        f"EC2 Console:
https://{REGION}.console.aws.amazon.com/ec2/home?region={REGION}#Instances:instanceId={iid}",
    ]
    return "\n".join(lines)

```

#

NOTIFICATION SENDERS

#

def post_webhook(url: str, payload: dict, label: str) -> bool:

```

if not url:
    print(f" [SKIP] {label}: no webhook URL configured")
    return False
data = json.dumps(payload).encode("utf-8")
req = urllib.request.Request(url, data=data, headers={"Content-Type": "application/json"},
method="POST")
try:
    with urllib.request.urlopen(req, timeout=10) as r:
        print(f" [OK] {label}: HTTP {r.getcode()}")
        return r.getcode() == 200
except urllib.error.HTTPError as e:
    print(f" [ERROR] {label}: HTTP {e.code} — {e.read().decode()[:200]}")
    return False
except Exception as e:
    print(f" [ERROR] {label}: {e}")
    return False

def send_sns_email(report: dict) -> bool:
    if not SNS_TOPIC_ARN:
        print(" [SKIP] Email: SNS_TOPIC_ARN not configured")
        return False
    top = "CRITICAL" if any(a["severity"] == "CRITICAL" for a in report["alerts"]) else "WARNING"
    subject = f"[{ENVIRONMENT.upper()}] {top} — EC2 {report['name']} ({report['instance_id']})"
    try:
        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject=subject[:100],
            Message=build_sns_email(report),
        )
        print(f" [OK] Email via SNS")
        return True
    except Exception as e:
        print(f" [ERROR] SNS email: {e}")
        return False

def notify(report: dict):
    """Send alert to all channels."""
    post_webhook(SLACK_WEBHOOK_URL, build_slack_message(report), "Slack")
    post_webhook(TEAMS_WEBHOOK_URL, build_teams_message(report), "Teams")
    send_sns_email(report)

```

```
#
```

```
# LAMBDA ENTRY POINT
```

```
#
```

```
def lambda_handler(event, context):
```

```
    """
```

```
    Triggered by EventBridge every 5 minutes.
```

```
    Checks all instances and sends alerts for any that are unhealthy.
```

```
    """
```

```
    if not INSTANCE_IDS:
```

```
        print("[ERROR] INSTANCE_IDS environment variable is empty. Nothing to check.")
```

```
        return {"statusCode": 400, "message": "No instance IDs configured"}
```

```
    print(f"[START] Checking {len(INSTANCE_IDS)} instances: {INSTANCE_IDS}")
```

```
    results = []
```

```
    alerted = 0
```

```
    for instance_id in INSTANCE_IDS:
```

```
        report = evaluate_instance(instance_id)
```

```
        results.append(report)
```

```
        if not report["healthy"]:
```

```
            print(f" [ALERT] {report['name']} has {len(report['alerts'])} issue(s) — sending notifications")
```

```
            notify(report)
```

```
            alerted += 1
```

```
        else:
```

```
            print(f" [OK] {report['name']} is healthy")
```

```
    summary = {
```

```
        "checked": len(INSTANCE_IDS),
```

```
        "healthy": len(INSTANCE_IDS) - alerted,
```

```
        "alerted": alerted,
```

```
        "timestamp": datetime.now(timezone.utc).isoformat(),
```

```
        "results": [
```

```
            {
```

```
                "id": r["instance_id"],
```

```
                "name": r["name"],
```

```
                "healthy": r["healthy"],
```

```
                "alerts": len(r["alerts"]),
```

```
    }  
    for r in results  
],  
}  
print(f"[DONE] {summary}")  
return {"statusCode": 200, "summary": summary}
```